Hyunsu Yang

Compositor

Nuke Tools & Compositing Workflow Portfolio

hyang.vfx@gmail.com | +1 416-500-8975 | Vancouver, BC
yanghyunsu.github.io

About

Nuke Compositor with 5+ years of feature film and episodic VFX experience, focused on CG/live-action integration, shot finaling, and shot-to-shot consistency. Strong eye for light, colour, grain, edge treatment, and photographic detail, with experience handling complex shots in fast-turnaround environments. Technical background in Python, BlinkScript, and Nuke tool development.

Technical Skills

Compositing:	Nuke, CG/live-action integration, AOVs, keying, tracking, roto/paint, colour matching, grain, edge treatment, shot finaling
Technical:	Python, BlinkScript, Nuke gizmos, Nuke Particle System, PySide2, Maya Python API, ShotGrid, Git, Linux

Projects in This Document

01 AOV Relight Toolkit

Nuke Gizmo / Blinkscript : A Nuke gizmo for AOV-based relighting

02 Risograph Shader

Nuke Gizmo / Blinkscript : 3-stage BlinkScript pipeline simulating Risograph printing

03 Particle-Based Lightning

Nuke Particle System /Expression : lightning generator built on Nuke's native particle system

04 Particle-Based Fire

Nuke Particle System : A controllable campfire built with Nuke's native particle system

Production Credits

Boxel Studio - Scooby-Doo: Origins, Nocturne
DNEG - Time Bandits, Haunted Mansion, For All Mankind S04
Digital Domain - Ant-Man and the Wasp: Quantumania, Black Panther: Wakanda Forever, She-Hulk: Attorney at Law
Switch VFX - What We Do in the Shadows S02&04, The Expanse S03&04, Star Trek Discovery S04, Chapelwaite, Sneakerella, Spiral: From the Book of Saw

PROJECT 01

AOV Relight Toolkit Nuke

Gizmo · Blinkscript · Python

OVERVIEW

- A Nuke gizmo for AOV-based relighting in compositing. Adds virtual lights to rendered scenes using Normal, Position, and Albedo passes.

PROCESS

- Single Nuke Gizmo with a BlinkScript kernel that computes Lambertian diffuse and Blinn-Phong specular lighting from AOV passes (Normal, Position, Albedo)
- Supports Directional and Point lights with linear/quadratic falloff, wrapped Lambert for soft terminators, and a Fresnel-style edge boost for grazing-angle highlights.
- 4 modes: Add to Beauty, Replace Lighting, Relight Only, and Source.

TECHNICAL DETAILS

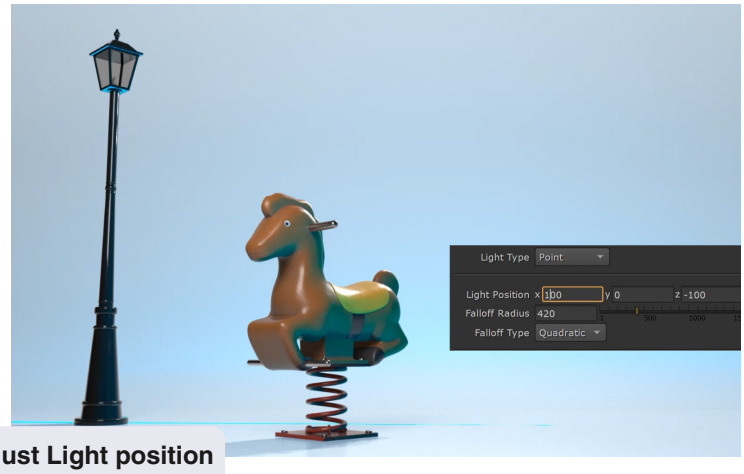
- **Blinn-Phong over Phong:** Phong highlights can cut off suddenly near the edges of objects. Blinn-Phong uses the direction halfway between the light and the camera (H = normalize(L + V)), so highlights stay smooth.
- **Wrapped Lambert:** diffuse = (N·L + wrap) / (1 + wrap). Lets the light wrap a little past the shadow line, so the shadow edge looks softer. Wrap 0 = normal Lambert, wrap 1 = light reaches almost all the way around the object.
- **Fresnel edge boost:** The kernel finds the camera direction for each pixel from the Position pass (V = normalize(cameraPos - P)), then makes specular stronger where the surface turns away from the camera, using (1 - N·V)^5. Idea from Alex Villabon's Fresnel tutorial, built into the kernel with the Normal and Position passes.
- **Camera auto-detect:** Reads Redshift EXR metadata (exr/rs/camera/transform) and extracts camera position from the 4×4 transform matrix (indices 12, 13, 14 = translation component).

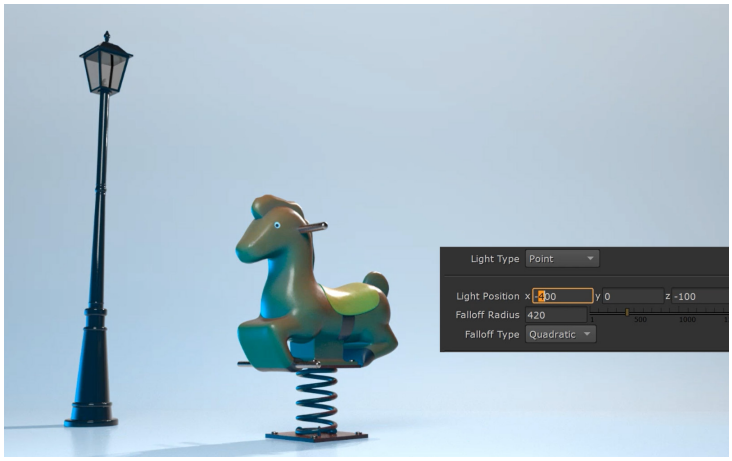
TOOLS & DELIVERY

Blinkscript | Python | Nuke Gizmo

DEMO

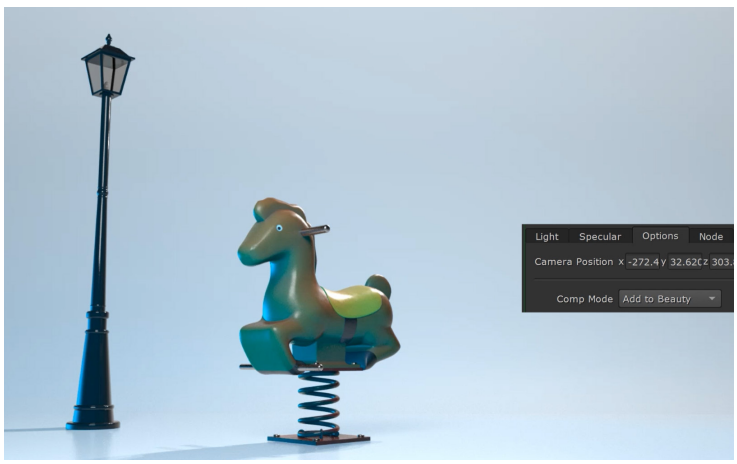
yanghyunsu.github.io/posts/aov-relight-toolkit.html





Adjust Light position





CompMode options

PROJECT 02

Risograph (Riso Printing) Shader

Nuke Blinkscript

OVERVIEW

- The goal was to recreate the look of Risograph printing inside Nuke using BlinkScript. I wanted a tool that could take any RGB image and simulate the full Riso printing process: ink separation, physical grain, and final print assembly.

PROCESS

- 3-stage Blinkscript pipeline that takes any RGB image and simulates the full Riso printing process:

1 : Ink Separation

Converts RGB into 3 ink coverage channels. Each pixel is compared against each ink color using cosine similarity combined with luminance matching.

2 : Ink Texture

Applies procedural fBm noise to each coverage channel. Each ink uses a different noise seed offset so grain patterns are independent. Coverage-based scaling: thin ink = heavy grain, thick ink = weak grain to matching real Riso ink absorption behaviour.

3 : Print Assembly

Composites final image using multiply blend for physically accurate ink layering. Uses eAccessRandom to read from offset coordinates, simulating misregistration between color plates. Adds paper base texture with 2-octave fBm noise.

TECHNICAL DETAIL

- Hue-based separation looked artificial. Combined **cosine similarity** (color direction in RGB space) with **luminance matching** to produce coverage values that feel like actual Riso ink plates rather than color filters.
- Cosine similarity (dot product of normalized color vectors) for ink separation
- fBm noise ➔ valueNoise ➔ fBm chain for procedural grain
- Multiply blend mode for physically-based subtractive ink compositing

FUTURE WORK

- Wrap all 3 kernels into a single Gizmo for one-node deployment
- Cryptomatte-based separation for per-object ink color assignment

TOOLS & DELIVERY

Blinkscript Nuke

DEMO

yanghyunsu.github.io/posts/risograph-shader.html

Stage-by-Stage Breakdown for Risograph print look in Nuke

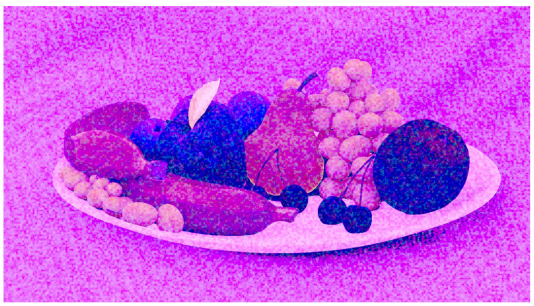




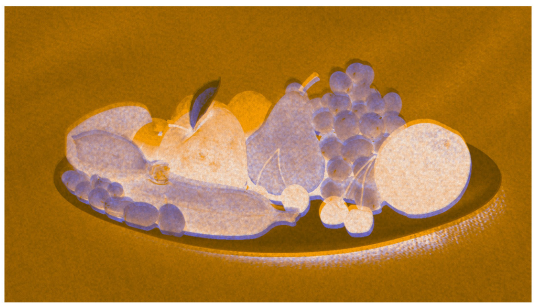
Original Input



Ink Separation



Ink Texture



Print Assembly

PROJECT 03

Particle-Based Lightning

Nuke Particle System, Expression

OVERVIEW

- A Nuke particle setup for timed lightning bursts, branching sparks, procedural sprite textures and force-driven motion

PROCESS

- **Tracer-trail construction:** a leader particle draws the bolt path under wind and layered turbulence during a pre-simulation window; ParticleSpawn trails stamp the path into a persistent bolt before the shot starts.
- **Cycle system**: modulo-driven emitter rate and trail lifetimes repeat the full appear -> flash -> fade choreography every N frames.
- **Per-bolt randomization** via cycle-indexed random(): strike position (Axis), bolt color, branch count.
- **Master control panel**: a NoOp exposing timing, thickness, branch density and master seed knobs

TECHNICAL DETAIL

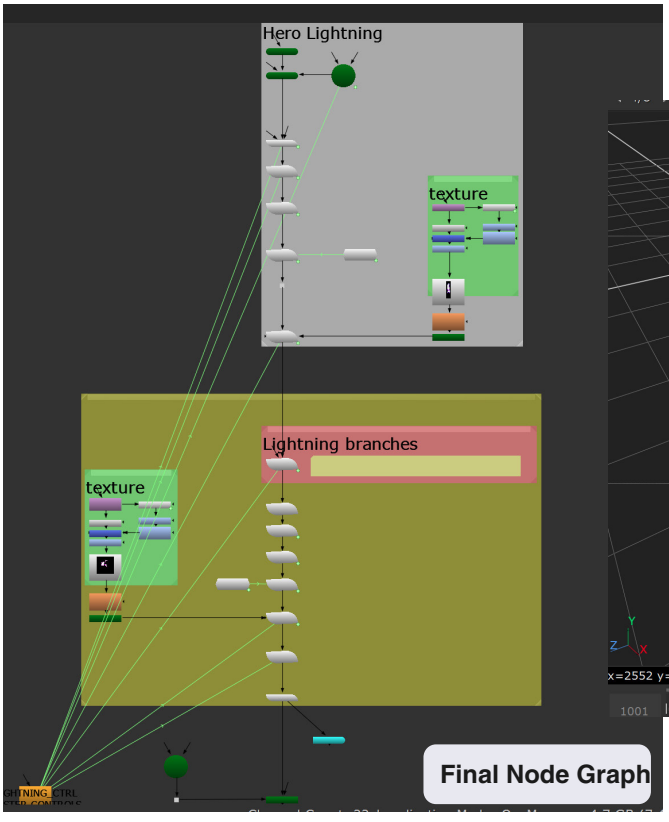
- **Remaining-lifetime clock**: Nuke's particle expression language has no frame or time variable, so global timing is encoded at spawn instead. Setting *lifetime = deathFrame - frame* on the spawn node makes every trail particle die on the same frame, which turns life - age into a synchronized countdown inside the expression.
- **Impact spike**: the size expression layers a 4-frame decaying spike on the base profile, landing each bolt at ~2× thickness on its exact impact frame.
- **Cycle-stable randomness:** *random(floor((frame - start)/period) + seed)* is constant within a cycle and re-rolls between cycles; spawn-time baking guarantees earlier bolts are never retroactively recolored or moved.
- **Knob access inside a sandboxed expression language**: ParticleExpression cannot reference external knobs directly. Its temp-variable slots do accept TCL [value] substitution, which is used to pipe panel knobs (thickness, impact timing) into the simulation expression.
- **Engine limitation found and designed around:** the life variable proved unreliable on 3rd-generation spawned particles; all expression logic is therefore constrained to 2nd-generation trails.

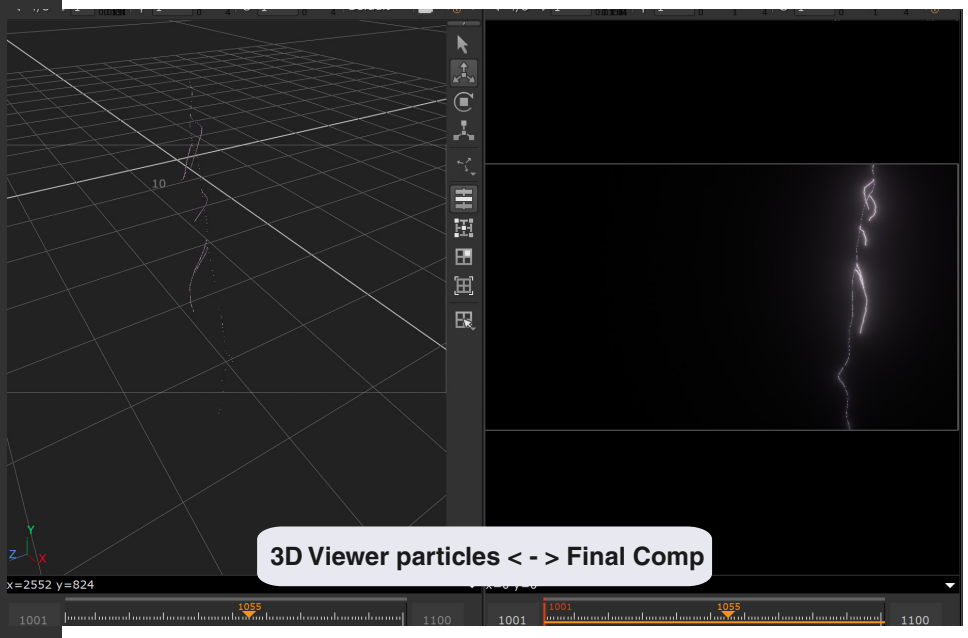
TOOLS & DELIVERY

Nuke Particle System

DEMO

yanghyunsu.github.io/posts/particles-lightning.html





PROJECT 04

Particle-Based Fire

Nuke Particle System, Expression

OVERVIEW

- A Nuke particle-system study exploring procedural fire motion, turbulence, directional force, and particle curve controls.

PROCESS

- **Procedural fire sprite**: animated turbulence × radial falloff; BlendMat set to plus/modulate so overlapping sprites accumulate brightness toward the flame core.
- **Geometry-based emission**: a noise-displaced, flattened sphere (normals forced to +Y) used as the emitter surface, so flames originate from a broad campfire base rather than a point.
- **Region-based shaping**: stacked sphere-region directional forces drive the updraft; box-region turbulence breaks the silhouette only inside the flame volume. Simple cylinder "log" geometry sits in the scene as occluders shaping the flame base.
- **Lifecycle**: ParticleCurve profiles drive size and opacity over particle age. The flame grows, peaks, and dissolves; embers run on a separate emitter with vortex + drag and their own curves.

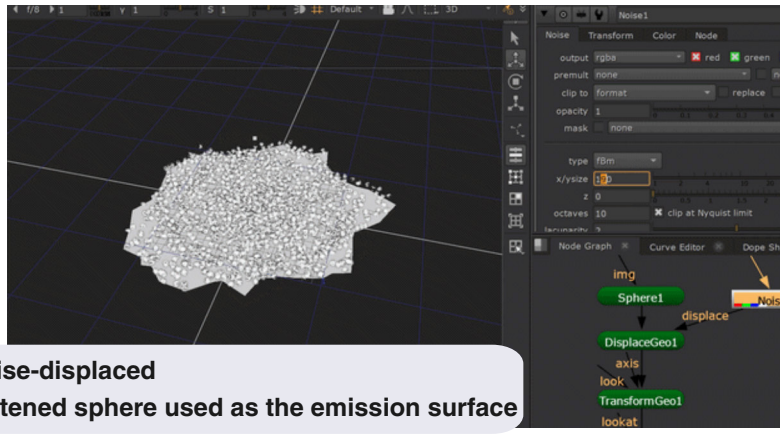
TECHNICAL DETAIL

- **Sub-frame force pulsing:** the directional-force regions are scaled by *1 - frame%1*. At whole frames this evaluates to a constant, but the solver runs 6 substeps per frame, so the expression is sampled at fractional frame times: the updraft region shrinks across each frame and resets on the next, producing the flame's natural flicker without keyframes.
- **Force-authored motion:** emission velocity is near zero (0.006 with 100% variation), so the flame's entire motion profile comes from the force fields rather than birth velocity.
- **Per-system solver budgets**: the flame body and the ember pass run as separate particle systems with their own ParticleSettings: 6 substeps per frame for the dense, force-heavy flame, 3 for the sparse embers.
- **Gradient-derived motion vectors:** Sobel kernels (Matrix nodes) extract x/y luminance gradients from the render; the field is normalized per pixel (h = hypot(r,g); r/h, g/h), sign-corrected, blurred, and copied into forward.u/v for VectorBlur; an optical-flow-style smear with no 3D motion vectors required.

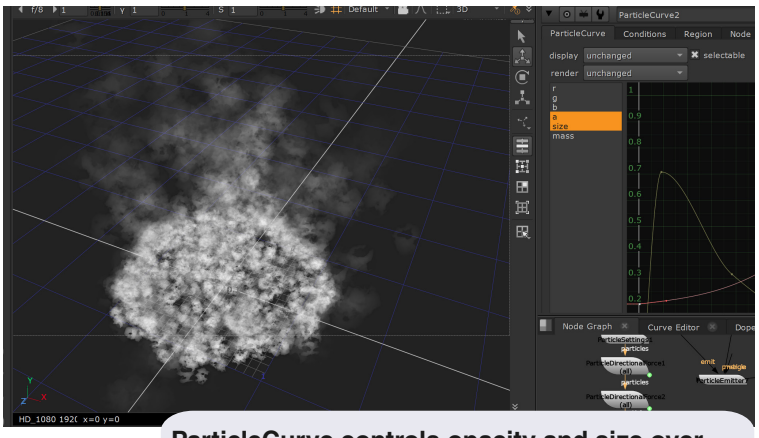
TOOLS & DELIVERY

Nuke Particle System **DEMO**

yanghyunsu.github.io/posts/fire-particle.html



Noise-displaced flattened sphere used as the emission surface



ParticleCurve controls opacity and size over particle age, shaping the flame from base to tip.



Final Comp